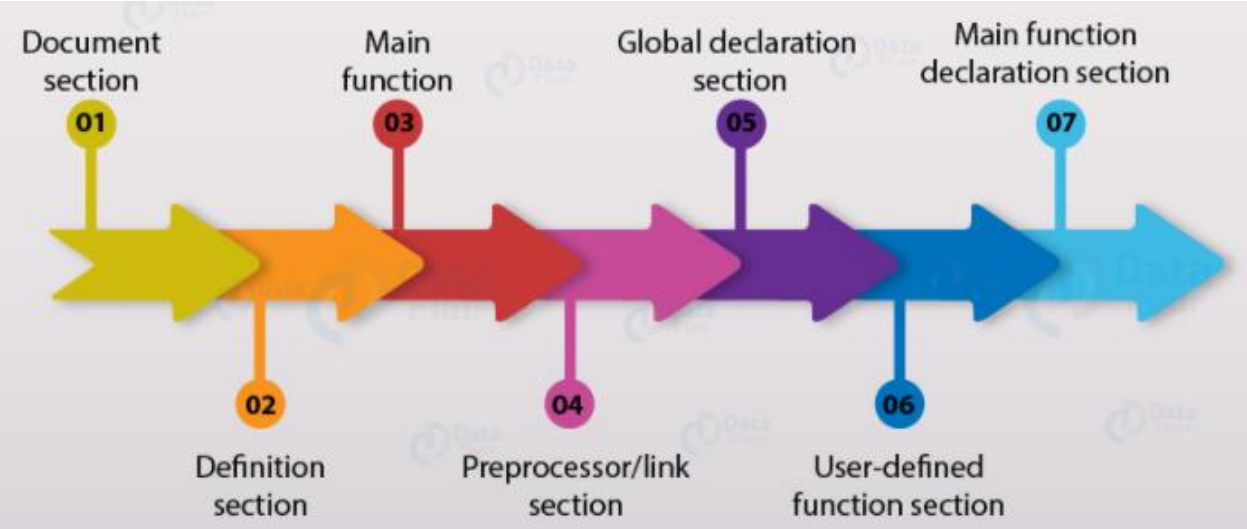
Lesson 7

Preview of Basic Structure of C Program:

When working in C it is important to be familiar with the various components/sections of a C program so as to better understand more advanced features of the C language. This is useful in developing more efficient code and properly understanding other people's code. Below is a brief description of the different sections that can be found in a C program.



The components of the basic structure of a C program consists of 7 parts

1. Documentation Section

It is the section in which you can give comments to make the program more interactive, and highlight important details pertaining to the program. The compiler won't compile this and hence this portion would not be displayed on the output screen.

2. Preprocessor directives Section

This section involves the use of header files that are to include necessarily program and other user defined macros.

3. Definition section

This section involves the variable definition and declaration in C.

4. Global declaration Section

This section is used to define the global variables to be used in the programs that means you can use these variables throughout the program.

5. Function prototype declaration section

This section gives the information about a function that includes, the data type or the return type, the parameters passed or the arguments.

6. Main function

It is the major section from where the execution of the program begins. The main section involves the declaration and executable section.

7. User-defined function section

When you want to define your function that fulfills a particular requirement, you can define them in this section.

An example of a simple code that has all the sections is as shown below.

```
/*
Documentation section
C programming structure
Author: **** */
/* Preprocessor section */
#include <stdio.h>

/* Global declaration, definition section */
int subtract = 0;

/* Function declaration section */
int all (int, int);

/* Main function */
int main (){
        printf ("This is a C program \n");
        subtract= all (25,10);
        printf ("Subtraction of the two numbers : %d \n", subtract);
        return 0;
}

/* User defined function */
int all (int x, int y) {
        return x-y; /* definition section */
}
```

Deeper Diver into Pre-Processor section:

Majority of the sections discussed above are well known but an important section that is very versatile is the **pre-processor** section.

In programming terminology, a preprocessor is nothing but a System Software that performs the processing of a high-level language before compilation by translating the source code written in a high-level language to object code written in the machine-level language, which is easily understood by the compiler.

In the C language, all of the preprocessor directives begin with a hash/pound (#) symbol. We know that in C we can use preprocessor directives anywhere in our program. But, it is preferable to use them at the beginning of the program. This enhances the readability of the code.

Preprocessor Types:

In C there are 4 basic types of preprocessors:

1: **File Inclusion:** File inclusion is responsible for instructing the compiler to include a specific file in the source code program. Depending on the type of file included, file inclusion is categorized into two types, namely:

Standard header files – These files refer to the pre-existing files, which convey a specific meaning to the compiler before the actual compilation has taken place.

User-defined files – The C language gives the programmer the provision to define their own header files in order to divide a complex code into small fragments.

2: Macros: In programming terminology, a macro is a segment of code that has the ability to provide the inclusion of header files and specifies how to map a replacement output sequence in accordance to a well-defined series of steps a particular input sequence. For example,

#define MAX 100

Here, the string MAX has the assigned constant value of 100. Also, MAX is called macro template and 100 is repressed to as macro expansion.

3: Conditional Compilation: Just like we use if-else statements for the flow of control over specific segments of code, in the same way, we use the concept of conditional compilation. Conditional compilation is a type of preprocessor that gives the programmer the power to control the compilation of particular segments of codes. It is done with the help of the 2 popular preprocessing commands, namely:

#ifdef

#endif

4: Other Directives: There are 2 more preprocessor directives in C apart from the ones already discussed. They are, namely:

#undef: As eccentric as it sounds, we use #undef directive to undefine the pre-existing, standard header or a user-defined header file.
#pragma: We use this type of preprocessor directive to enable or disable certain features. It is important to note that #pragma varies from compiler to compiler and can be mainly seen implemented in PIC microcontrollers.

Summary Table:

Preprocessor	Elucidation
#include	**Used to insert a specific header from a file.**
#define	**Used as a replacement of a preprocessor macro.**
#ifdef	**Used when dealing with conditions. If the macro is defined, it returns true.**
#endif	**Used to close the preprocessor directive in accordance with a given condition.**
#undef	**Used to undefine a standard or user-defined header.**
#pragma	**Used to enable and disable certain features.**
#ifndef	**If the macro is not defined, it returns true.**
#if	**Used to check if the condition is true in compile time.**
#else	**Used to check the next condition if #if proves to be false in compile time.**
#elif	**Used as a combination of #else and #if.**
#error	**Used to print error on stderr.**

For some examples of pre-processor implementations use the following URL:

https://github.com/ExploreEmbedded/Code-Libraries/tree/master/CodeLibraries/AVR

Code Automation: Macros and Functions

Macros:

In computer science, a macro is a rule or pattern or section of code that specifies how a certain input should be mapped to a certain input should be mapped to a replacement output. In C, the Macro substitution is a mechanism which provides a string substitution. It can be achieved through.

Syntax:

```
#define Label Expression
```

It is used to replace the label with the expression of macro definition, before the execution of the program. The first object may be function type or an object. This can be used

Types of Macros in C Programming
In C, Macros are broadly classified into two distinct types. They are namely:

Object-like Macros: It appears to be a symbolic constant. It can be termed as an alternative way to define an identifier used to represent constant expressions. The simplest example would be:
#define PI 3.14

Function-like Macros: It is an expression, used to perform a particular operation. It is an alternative way to define a function. A simple example would be:
#define RECTANGLE(l,b) l*b

An example of a simple code that has all the sections is as shown below.

```
#include<stdio.h>

#define RECTANGLE(l,b) l*b

int main(){
 int length = 3, breadth = 4;
 int area = RECTANGLE(length,breadth);
 printf("The area is: %d\n", area);
 return 0;
}
```

An example of a simple code that uses a multi-line function-like macro to swap 2 numbers using an XOR

```
// We use a trick involving exclusive-or to swap two variables
#define SWAP(a, b)  a ^= b; b ^= a; a ^= b;

int main() {
  int x = 10;
  int y = 5 ;
  // works OK
  SWAP(x, y);
   return 0;
}
```

```
// An alternative way of writing a multi-line macro

#define SWAP(a, b)  {                    \
                        a ^= b;          \
                        b ^= a;          \
                        a ^= b;          \
                    }
int main() {
  int x = 10;
  int y = 5 ;
  // works OK
  SWAP(x, y);
   return 0;
}
```

Functions: Key concepts (What is a function, why do we need functions, how do functions operate)

Subroutines/Functions are one of the most basic forms of code reuse. If you have a piece of code that needs to be run at different times or with different data, a *function* lets you isolate that code and run it ("call it") from any other section of code, at any time. In fact, a *function* can even call itself, and this is known as **RECURSION** which we will discuss in depth later.

How functions work:

The essence of a *function* call instruction is that it is a jump (jumping to the *function* code), but a jump that remembers where it came from. By remembering the address that it came from, at the end of the *function* it can jump back to where it came from so the calling code can continue. To be more precise, a *function* call instruction is a jump instruction which saves the address of the instruction following the *function* call instruction, not the address of the *function* call instruction itself. It is the next instruction that needs to be executed after the *function* returns. If the *function* returned to the *function* call instruction, the program would find itself in an endless loop, calling the *function* without end.

To return from a *function*, as noted, the PC must be loaded with the address that was saved when the subroutine was called. And as mentioned, this address could be in memory on a stack, or it could be in a link register. Many CPUs have a special instruction called a "return" instruction that fetches the address and sticks it into the PC. Some CPUs just use a standard move instruction to move the saved address into the PC. But the result is the same in any case - the program continues to execute where it left off.

Functions Application:

As mentioned before a function is a group of statements that together perform a task. Every C program has at least one function, which is main(), and all the most trivial programs can define additional functions. You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task.

A function declaration tells the compiler about a function's name, return type, and parameters. A function definition provides the actual body of the function.

The C standard library provides numerous built-in functions that your program can call. For example, strcat() to concatenate two strings, memcpy() to copy one memory location to another location, and many more functions.

1. *Defining a Function:*

The general form of a function definition in C programming language has the following syntax:

```
return_type function_name( parameter list ) {
   body of the function
}
```

A function definition in C programming consists of a function header and a function body. Here are all the parts of a function

Return Type – A function may return a value. The return_type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword void.

Function Name – this is the actual name of the function. The function name and the parameter list together constitute the function signature.

Parameters – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.

Function Body – the function body contains a collection of statements that define what the function does.

Example Code:

```
/* function returning the max between two numbers */
int max(int num1, int num2) {
   /* local variable declaration */
   int result;
   if (num1 > num2){
      result = num1;
   }
   else{
      result = num2;
   }
   return result;
}
```

```
Return type - int
Function name - max
Parameters - int num1, int num2
Function body - Checking which number between num1 and num2 is larger using the if statement
```

2. *Function Declarations:*

A function declaration tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately. A function declaration has the following parts –

```
return_type function_name( parameter list );
```

For the above defined function max(), the function declaration is as follows – `int max(int num1, int num2);`

Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration – `int max(int, int);`

Function declaration is required when you define a function in one source file and you call that function in another file. In such case, you should declare the function at the top of the file calling the function.

3. *Calling a Function*

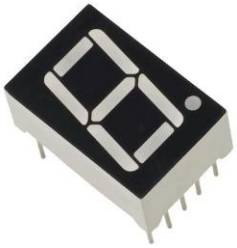
While creating a C function, you give a definition of what the function has to do. To use a function, you will have to call that function to perform the defined task. To call a function, you simply need to pass the required parameters along with the function name, and if the function returns a value, then you can store the returned value.

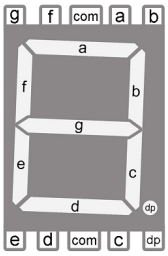
Example Code:

```
#include <stdio.h>

int max(int num1, int num2); /* function declaration */

int main () {

   int a = 100;    /* local variable definition */
   int b = 200;
   int ret;

   ret = max(a, b);/* calling a function to get max value */
   printf( "Max value is : %d\n", ret );

   return 0;
}

/* function returning the max between two numbers */
int max(int num1, int num2) {
   int result;         /* local variable declaration */
   if (num1 > num2) result = num1;
   else result = num2;
   return result;
}
```

7-segment LED Display

So far we have handled simple single pin output devices such as an LED but now we move on to more complex methods of display. To begin with, we will consider a typical 7-segment LED displays are formed by LED segments. It is basically used to display numerical values from 0 to 9, and other alphanumeric characters (A, B, C, D, E, and F). One more segment is also present there which is used as decimal point. Several of these segment LED can be combined to form a digital display.





Connection Types

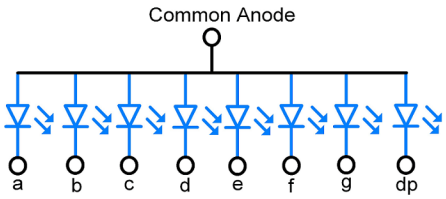
In 7-segment displays there are two types, common anode and common cathode.

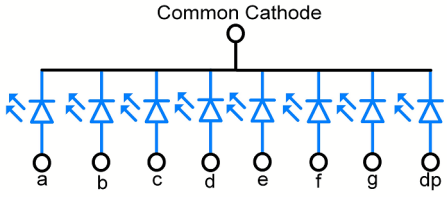
1. **Common Anode (CA)**

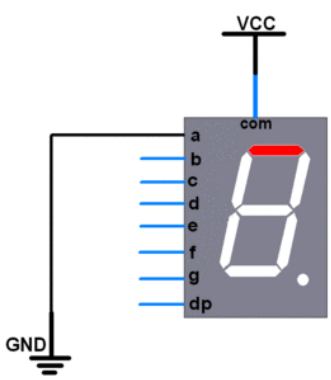
In common anode display, all anode pins are connected together to VCC and LEDs are controlled via cathode terminals. It means to turn ON LED (segment), we have to make that cathode pin logic LOW or Ground.

2. **Common Cathode (CC)**

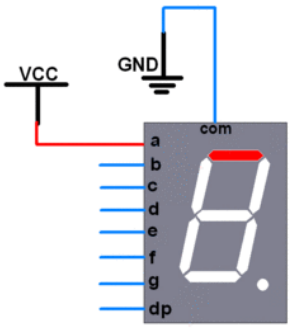
In common cathode display, all cathode pins are connected together and led are controlled via anode terminal. It means to turn ON LED (segment), we have to apply proper voltage to the anode pin.







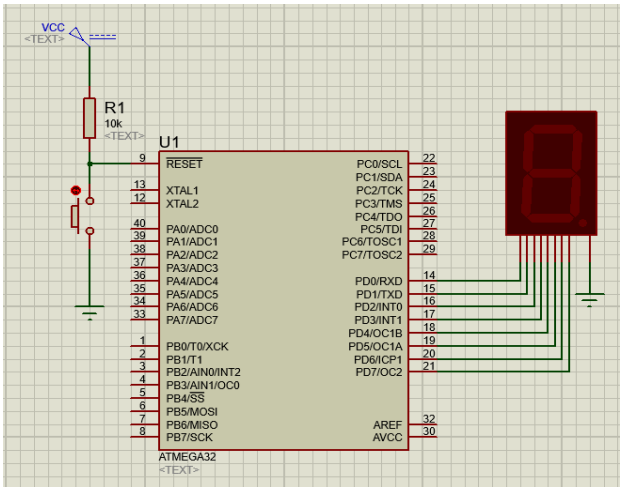
Common Anode



Common Cathode

Figure 1: Connection circuits for common anode and common cathode connections

Circuit Implementation:



	Binary	HEX
0	00111111	0x3F
1	00000110	0x06
2	01011011	0x5b
3	01001111	0x4f
4	01100110	0x66
5	01101101	0x6d
6	01111101	0x7d
7	00000111	0x07
8	01111111	0x7f
9	01101111	0x6f

Example Code:

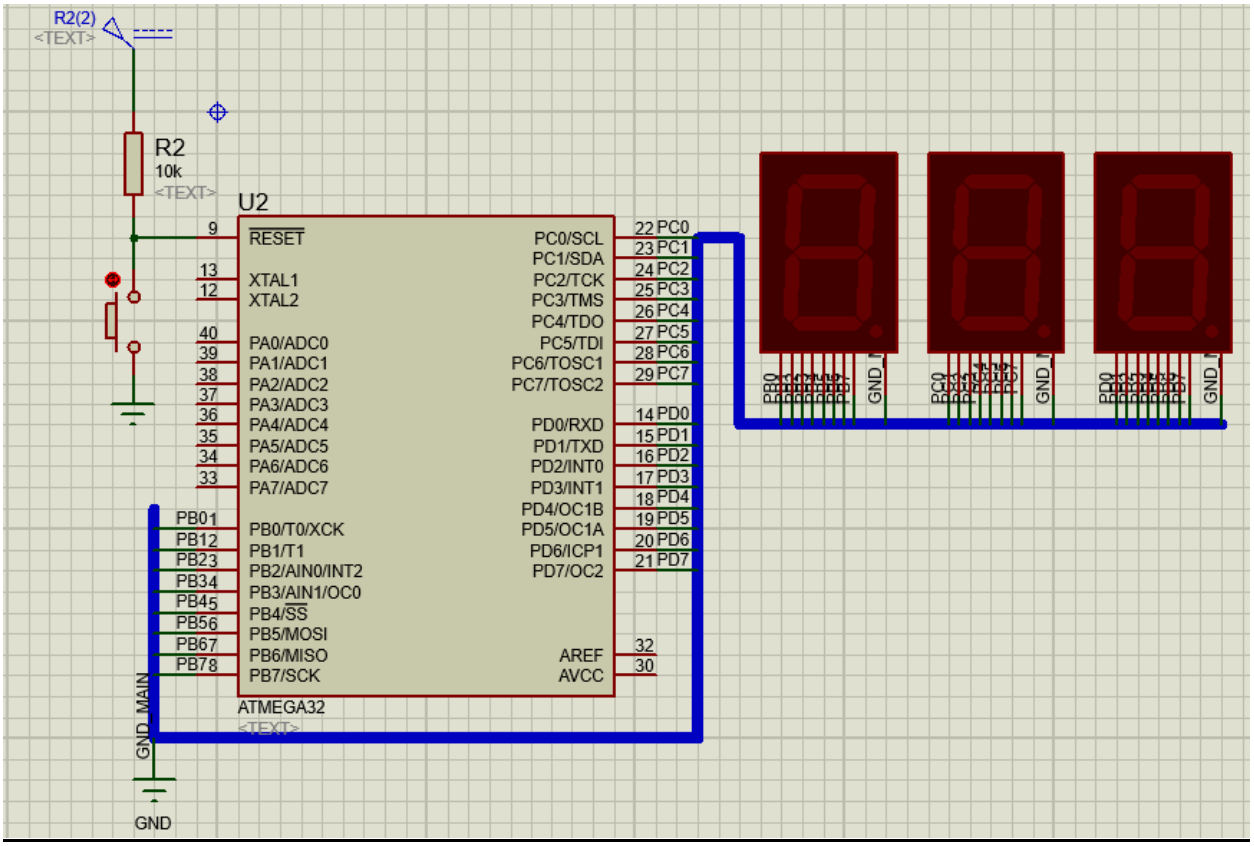
```c
/*
 * 7_segment.c
 */
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#define LED_Dir DDRD          /*define LED Directory*/
#define LED_Port PORTD        /*define LED PORT*/

void display(int digit);/*function declaration*/

int main(void)
{
  /*initialization*/
  LED_Dir |= 0xff;
  LED_Port = 0xff;

  while (1){
    for(int i=0;i<10;i++){
      display(i);      /* Function call */
      _delay_ms(1000);/* wait for 1 second */
    }
  }
}

/*Function definition*/
void display(int digit){
  /*We light each LED one at a time*/
  /* write hex value for Common Cathode display from 0 to 9 */
  char numbers[] = {0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f};
  if(digit < 10 ){
    LED_Port = numbers[digit];
  }
}
```

Step 3 – Compile, upload and simulate the code in Proteus

WHAT NEXT?

Now that you have an understanding of manipulating a single 7-segment LED, try and construct a simple count-down timer that can count down 2 minutes. Use 3, 7-segment LEDS, and include the use of the decimal place light on the first segment.

Circuit Implementation:



Above is the circuit implementation of a 3 segment count down timer. In the above circuit we use what is known as a bus to make the circuit diagram as neat as possible. To use a bus, you first draw the bus path and then connect all the wire lines and connect them to the bus line. Once connected to the bus, the wires are connected by labelling the using the same labels. i.e. if PORTB pins are to be used for the first 7-segment LED, then the wires from PORTB i.e. label PB0, PB1… PB7 are the same labels used form the 1st LED, i.e. PB0, PB1 … PB7.

Once all the wire connections are made, use the template below to write a countdown timer AVR code for the circuit.

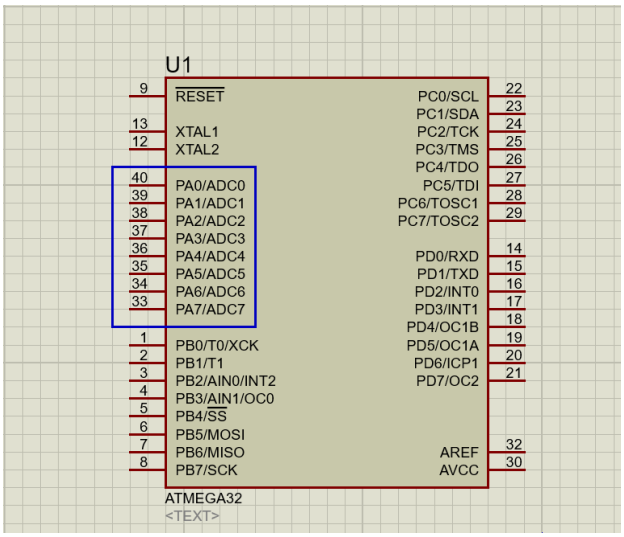
Sample Code:

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdbool.h>

#define SEG1_Dir DDRB            /*define SEG1 Directory*/
#define SEG1_Port PORTB          /*define SEG1 PORT*/
#define SEG2_Dir DDRC            /*define SEG2 Directory*/
#define SEG2_Port PORTC          /*define SEG2 PORT*/
#define SEG3_Dir DDRD            /*define SEG3 Directory*/
#define SEG3_Port PORTD          /*define SEG3 PORT*/

void display_1(int digit);       /*Function declaration*/
void display_2(int digit);       /*Function declaration*/
void display_3(int digit);       /*Function declaration*/
void flash_zero(void);           /*Function declaration*/
char numbers[] = {0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x00};

int main(void)
```

```
{
        /*initialization*/
        SEG1_Dir |= 0xff;
        SEG1_Port = 0xff;
        SEG2_Dir |= 0xff;
        SEG2_Port = 0xff;
        SEG3_Dir |= 0xff;
        SEG3_Port = 0xff;
        bool active = true;

        /* setup the minutes and seconds values*/
        int minutes = 1;
        int seconds = 10;
        int count_down = (minutes*60)+seconds;

        /*main loop*/
        while (1){
                if(active){
                        for(int i=count_down;i > 0;i--){
                                int secs = i%60;
                                int mins = i/60;
                                display_3(secs%10);
                                display_2(secs/10);
                                display_1(mins);        /* Function to display a digit */
                                _delay_ms(1000);        /* wait for 1 second */
                        }
                        active = false;
                }
                else{
                        flash_zero();
                }
        }
}

/*Function Definitions*/
void display_1(int digit){
        char output = numbers[digit] | 0x80 ;
        SEG1_Port = output;
}

void display_2(int digit){
        SEG2_Port = numbers[digit];
}

void display_3(int digit){
        SEG3_Port = numbers[digit];
}

void flash_zero(void){
        display_1(0);
        display_2(0);
        display_3(0);
        _delay_ms(500);/* wait for 1 second */
        display_1(10);
        display_2(10);
        display_3(10);
        _delay_ms(500);/* wait for 1 second */
}
```

Lesson 8

ADC (Analog to Digital converter) is most widely used feature of micro-controllers. It is a feature mainly used for data acquisition. This means reading analogue values from other devices such as potentiometers, sensors etc. In AVR ATmega series normally 10-bit ADC is inbuilt in the controller. Taking a look at the ATmega32, we find it supports an eight ADC channels, means we can connect eight analog inputs at a time. ADC channel 0 to channel 7 are present on PORTA. i.e. Pin no.33 to 40.



The controller has 10 bit ADC, means we will get digital output 0 to 1023. i.e. When input is 0V, digital output will be 0V & when input is 5V (and Vref=5V), we will get highest digital output corresponding to 1023 steps, which is 5V. So controller ADC has 1023 steps and

- Step size with Vref=5V : 5/1023 = 4.88 mV.
- Step size with Vref=2.56 : 2.56/1023 = 2.5 mV.

So Digital data output will be Dout = Vin / step size.

ATmega32 ADC specifications:

- It is 10-bit ADC
- Converted output binary data is held in two special function 8-bit register ADCL (result Low) and ADCH (result High).
- ADC gives 10-bit output, so (ADCH : ADCL) only 10-bits are useful out of 16-bits.
- We have options to use this 10-bits as upper bits or lower bits.
- We also have three options for reference voltage (Vref):1. AVcc -analog Vcc, 2. External Aref. Pin, 3. Internal 2.56v
- Total conversion time depends on crystal frequency and ADPS0 : 2 (frequency devisor)
- If you decided to use AVcc or Vref pin as ADC voltage reference, you can make it more stable and increase the precision of ADC by connecting a capacitor between that pin and GND.

ADC Registers: Consulting the Data Sheet

In AVR ADC, we need to understand four main register -

1. ADCH: Holds digital converted data higher byte
2. ADCL: Holds digital converted data lower byte
3. ADMUX: ADC Multiplexer selection register
4. ADCSRA: ADC Control and status register
5. ADCH : ADCL register

First two register holds the digital converted data, which is 10-bit.

ADMUX Register

7	6	5	4	3	2	1	0
REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0

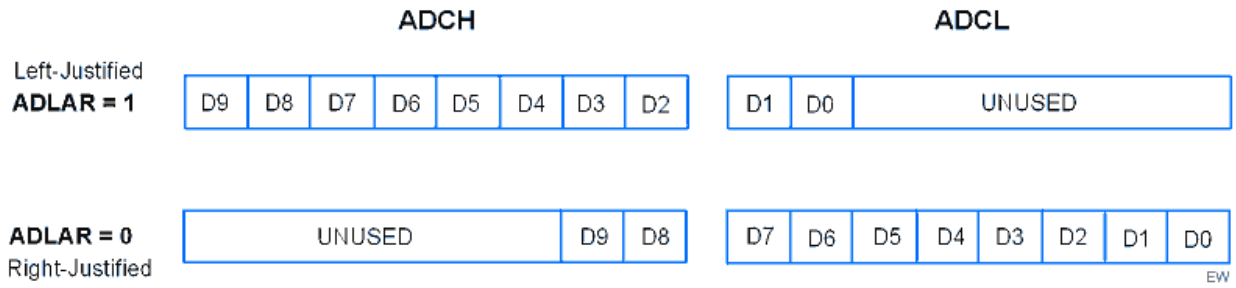
ADMUX Register

Bit 7 : 6 – REFS1 : 0: Reference Selection Bits, these bits allow the reference voltage selection for ADC

REFS1	REFS0	Vref ot ADC
0	0	AREF pin
0	1	AVCC pin i.e. VCC 5v
1	0	Reserved
1	1	Internal 2

Bit 5 – ADLAR : ADC Left Adjust Result

Use 10-bits output as upper bits or lower bits in ADCH & ADCL.



Bits 4 : 0 – MUX4 : 0 : Analog Channel and Gain Selection Bits

We can select input channel ADC0 to ADC7 by using these bits. These bits are also used to select comparator (inbuilt in AVR) inputs with various gain. We will cover these comparator operations in other part.

Selecting channel is very easy, just put the channel number in MUX4

Suppose you are connecting input to ADC channel 2 then put 00010 in MUX4

Suppose you are connecting input to ADC channel 5 then put 00101 in MUX4

ADCSRA Register:

7	6	5	4	3	2	1	0
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0

Bit 7 – ADEN : ADC Enable

Writing one to this bit enables the ADC. By writing it to zero, the ADC is turned off. Turning the ADC off while a conversion is in progress, will terminate this conversion

Bit 6 – ADSC : ADC Start Conversion

Writing one to this bit starts conversion.

Bit 5 – ADATE : ADC Auto Trigger Enable

Writing one to this bit, results in Auto Triggering of the ADC is enabled

Bit 4 – ADIF : ADC Interrupt Flag

This bit is set when an ADC conversion completes and the Data Registers are updated.

Bit 3 – ADIE : ADC Interrupt Enable

Writing one to this bit, the ADC Conversion Complete Interrupt is activated.

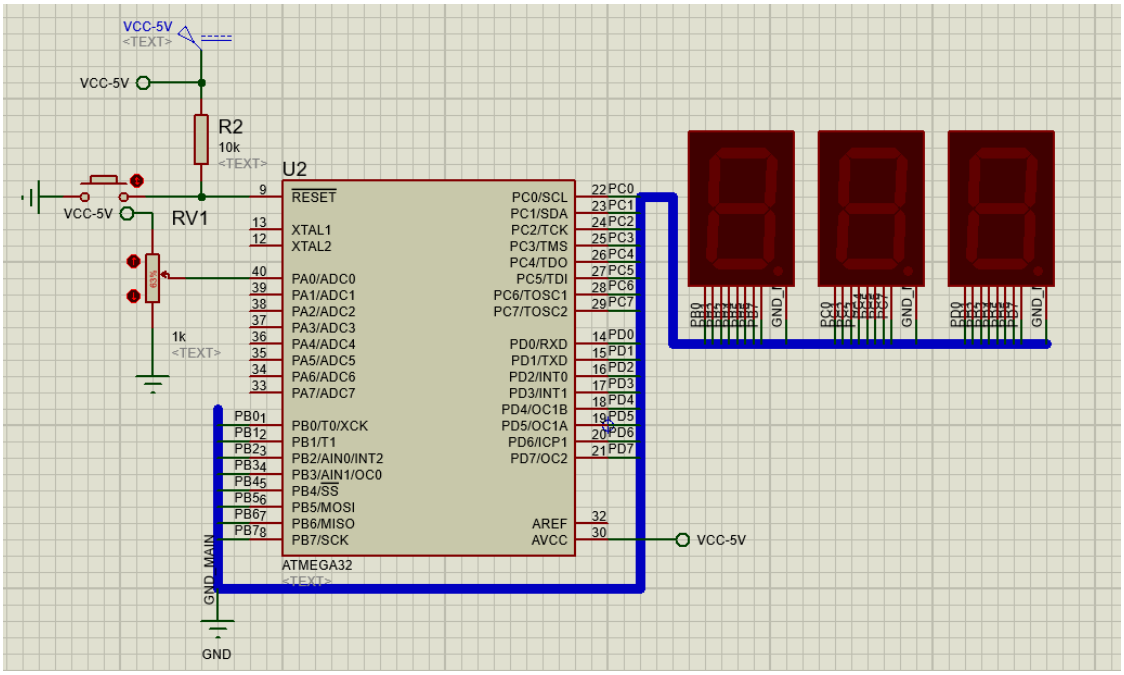
Bits 2 : 0 – ADPS2 : 0 : ADC Prescaler Select Bits

These bits determine the division factor between the XTAL frequency and the input clock to the ADC

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

We can select any divisor and set frequency Fosc/2, Fosc/4 etc. <u>for ADC. But in AVR, ADC requires an input clock frequency less than 200KHz for max. accuracy. So we have to always take care about not exceeding ADC frequency more than 200KHz.</u> Suppose your clock frequency of AVR is 8MHz, then we must have to use devisor 64 or 128. Because it gives 8MHz/64 = 125KHz, which is lesser than 200KHz.

Circuit Implementation:



Code Implementation:

```c
/*
 * ADC_example.c
 * Created: 09-Jul-19 10:14:58 AM
 * Author : Michael Mureithi
 */
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <stdbool.h>

#define SEG1_Dir DDRB            /*define SEG1 Directory*/
#define SEG1_Port PORTB          /*define SEG1 PORT*/
#define SEG2_Dir DDRC            /*define SEG2 Directory*/
#define SEG2_Port PORTC          /*define SEG2 PORT*/
#define SEG3_Dir DDRD            /*define SEG3 Directory*/
#define SEG3_Port PORTD          /*define SEG3 PORT*/

void DISPLAY_Init();             /*Function declaration*/
void display_1(int digit);       /*Function declaration*/
void display_2(int digit);       /*Function declaration*/
void display_3(int digit);       /*Function declaration*/

char numbers[] = {0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x00};
/*Function declaration*/
void ADC_Init(){
        DDRA = 0x0;              /* Make ADC port as input */
        ADCSRA = 0x87;           /* Enable ADC, fr/128 */
        ADMUX = 0x40;            /* Vref: Avcc */
}

int ADC_Read(int channel){
        int Ain,AinLow;

        ADMUX = ADMUX|(channel); /* Set input channel to read */
        ADCSRA |= (1<<ADSC);             /* Start conversion */
        while((ADCSRA & (1<<ADIF)) == 0);/* Monitor end of conversion interrupt */
        _delay_us(10);

        AinLow = (int)ADCL;      /* Read lower byte*/
        Ain = (int)ADCH*256;     /* Read higher 2 bits and Multiply with weight */
        Ain = Ain + AinLow;
        return(Ain);             /* Return digital value*/
}

int main(){
        int value;
        int hunds, tens, ones;
        ADC_Init();
        DISPLAY_Init();

        while(1)
        {
                value = ADC_Read(0);     /* Read ADC channel 0 */
                value = value/10;
                hunds = value/100;
                value %= 100;
                tens = value/10;
                ones = value%10;

                display_1(hunds);
                display_2(tens);
                display_3(ones);
        }
        return 0;
}
```

```c
/*Function declaration*/
void DISPLAY_Init(){
        /*initialization*/
        SEG1_Dir |= 0xff;
        SEG1_Port = 0xff;
        SEG2_Dir |= 0xff;
        SEG2_Port = 0xff;
        SEG3_Dir |= 0xff;
        SEG3_Port = 0xff;
}

void display_1(int digit){
        SEG1_Port = numbers[digit];
}

void display_2(int digit){
        SEG2_Port = numbers[digit];
}

void display_3(int digit){
        SEG3_Port = numbers[digit];
}
```

Lesson 9

Timer in AVR ATmega32:

Timers is a feature in micro-controllers which enables the precise counting of time. In a lot of application, we require to process data, inputs (sensor data) or outputs (variable voltage – PWM), generate waveforms and delays, count events etc. with precise timing. Thus we will look at how to interface with onboard timers that are available on the Atmega32.
In AVR ATmega32, there are three timers:

- **Timer0**: 8-bit timer
- **Timer1**: 16-bit timer
- **Timer2**: 8-bit timer

The number of bits indicates the maximum count which the timer can handle i.e. 8-bits = 256, 16-bit = 65536. If a timer goes above the max count value, it overflows. The status of the timer is indicated by its registers and flags. The typical registers and flags are as shown below:

TCNTn: Timer / Counter Register
Every timer has timer/counter register. It is zero upon reset. We can access value or write a value to this register. It counts up with each clock pulse.
TOVn: Timer Overflow Flag
Each timer has Timer Overflow flag. When timer overflows, this flag will get set.
TCCRn : Timer Counter Control Register
This register is used for setting the modes of timer/counter.
OCRn : Output Compare Register
The value in this register is compared with the content of the TCNTn register. When they are equal, OCFn flag will get set. It is often used to set a limit/set point value for the timer.
Let us look at Timer0 as an example to understand its operation.

1. **TCNT0: Timer / Counter Register:** It is an 8-bit register. It counts up with each pulse.
2. **TCCR0: Timer / Counter Control register 0**: This is an 8-bit register used for the operation mode and the clock source selection.

7	6	5	4	3	2	1	0
FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00

Bit 7- FOC0: Force compare match: Write only bit, which can be used while generating a wave. Writing 1 to this bit causes the wave generator to act as if a compare match has occurred.

Bit 6, 3 - WGM00, WGM01: Waveform Generation Mode

WGM00	WGM01	Timer0 mode selection bit
0	**0**	Normal
0	**1**	CTC (Clear timer on Compare Match)
1	**0**	PWM, Phase correct
1	**1**	Fast PWM

Bit 5 - 4: COM01:00: Compare Output Mode: These bits control the waveform generator. We will see this in the compare mode of the timer.
Bit 2 – 0: CS02:CS00: Clock Source Select: These bits are used to select a clock source. When CS02: CS00 = 000, then timer is stopped. As it gets value between 001 ;to 101, it gets clock source and starts as the timer.

CS02	CS01	CS00	Description
0	**0**	**0**	No clock source (Timer / Counter stopped)
0	**0**	**1**	clk (no pre-scaling)
0	**1**	**0**	clk / 8
0	**1**	**1**	clk / 64
1	**0**	**0**	clk / 256
1	**0**	**1**	clk / 1024
1	**1**	**0**	External clock source on T0 pin. Clock on falling edge
1	**1**	**1**	External clock source on T0 pin. Clock on rising edge.

3. **TIFR: Timer Counter Interrupt Flag register:** This register contains the overflow flag and output compare flags for timer 0, 1 and 2.

7	6	5	4	3	2	1	0
OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0

Bit 0 - TOV0: Timer0 Overflow flag, **0 =** Timer0 did not overflow**1 =** Timer0 has overflown (going from 0xFF to 0x00)
Bit 1 - OCF0: Timer0 Output Compare flag, **0 =** Compare match did not occur, **1 =** Compare match occurred
Bit 2 - TOV1: Timer1 Overflow flag
Bit 3 - OCF1B: Timer1 Output Compare B match flag
Bit 4 - OCF1A: Timer1 Output Compare A match flag
Bit 5 - ICF1: Input Capture flag
Bit 6 - TOV2: Timer2 Overflow flag
Bit 7 - OCF2: Timer2 Output Compare match flag

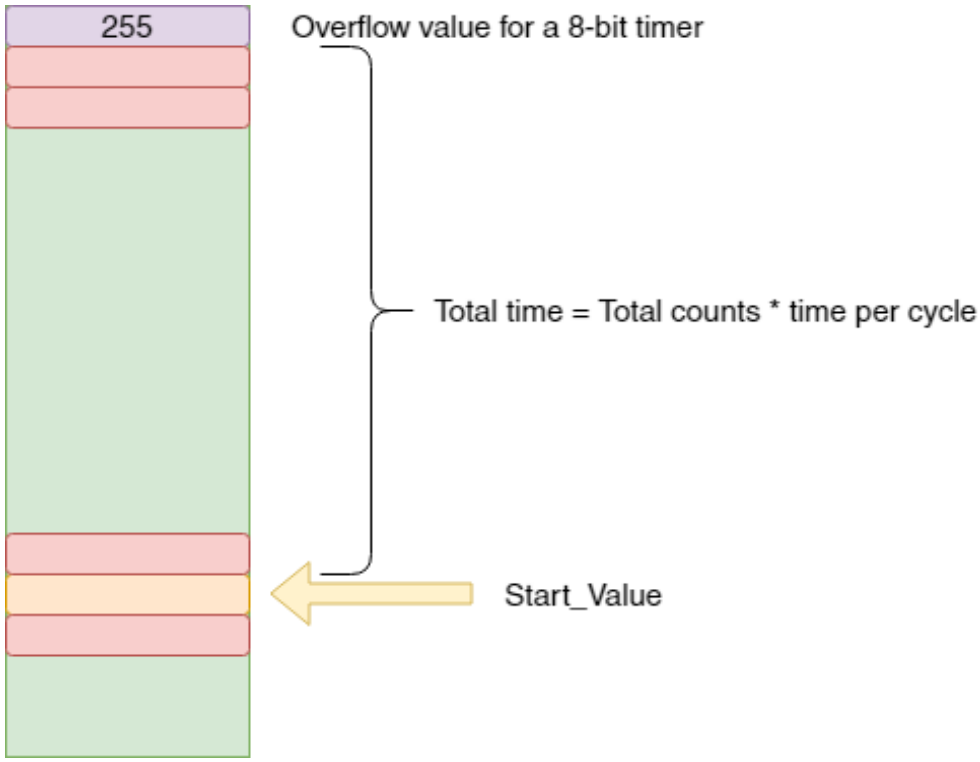
<u>Typical Timer Operation</u>
Now that we are conversant with the necessary registers in order to control the timers look at take a deeper took a typical timer application and what is really going on with an example.
The general, the microcontroller operates at a certain frequency (Fosc) e.g. 1MHz, 4MHz, 8MHz etc. This means each activity/microcontroller cycle takes approximately:

$$Time\ cycle = \frac{1}{F_{osc}}$$

If Fosc = 8MHz, the Time cycle $= \frac{1}{F_{osc}} = \frac{1}{8*10^6} = 0.125\mu s$
So the timer counts a certain number of these cycles and once it reaches the maximum number of cycles in the register, it overflows and the timer period is up. The overflow value of an 8 bit timer is 255. If we have a start value of let say 10, the

timer will count from 10, and after every time cycle i.e. 0.125µs increment the register by 1, up until it reaches the max value(255) after which it overflows and the timer repeats. Thus the time from start to over flow is

$$Total\ time = (255 - 10) * time\ cycle = 245 * 0.125 = 30.625\mu s$$



Prescaler:

From the above calculation we can observe that with an 8 bit timer, the max count value is 255. With an operation frequency of 8MHz, the maximum time period that we can count is$255 * 0.125\mu s = 31.875\mu s$. We can notice that this is a somewhat limited time range. In order to expand our time range, we use Prescaler.

In essence with the prescaler, the time per cycle calculation becomes:

$$Time\ cycle = \frac{1}{F_{osc}} * Prescaler$$

The table below shows the time cycles and max timer counts for a microcontroller with a Fosc = 8MHz and for various prescaler values range from 1, 8, 64, 256, 1024.

Prescaler Value	**1**	**8**	**64**	**256**	**1024**
Time Cycle	0.125 µs	1 µs	8 µs	32 µs	128 µs
Max time (255)	32 µs	255 µs	2 ms	8 ms	32 ms

Example:

Setup a timer0 to create a time delay of 20 µs with operating with a Fosc of 8MHz. (3mks)

Soln:

$$Time\ cycle = \frac{1}{F_{osc}} = \frac{1}{8 * 10^6} = 0.125\mu s$$
$$Timer\ counts = \frac{20\mu s}{0.125\mu s} = 160\ counts$$
$$Start\ Value = 255 - 160 = 95\ counts$$

In Hexadecimal,

$$Start\ Value = 95 = 0x5F$$

Code Implementation:

Steps to Program Delay using Timer0
1. Load the TCNT0 register with start value (i.e. TCNT0 = 0x5F).
2. For normal mode and pre-scaler option of the clock, set the value in TCCR0 register. As soon as clock prescaler value gets selected, the timer / counter starts to count, and each clock tick causes the value of timer / counter to increment by 1.
In this case where the operation is in normal mode with a prescaler of 1, TCCR0 is loaded with the value (0x01).

TCNT0	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
	0	0	0	0	0	0	0	1

TCCR0 = 0x01
3. Timer keeps counting up, so keep monitoring for timer overflow i.e. TOV0 (Timer0 Overflow) flag to see if it is raised.
4. Stop the timer by putting 0 in the TCCR0 i.e. the clock source will get disconnected and the timer / counter will get stopped.
5. Clear the TOV0 flag. Note that we have to write 1 to the TOV0 bit to clear the flag.
6. Return to the main function.

```
#include <avr/io.h>
void timer_delay();

int main(void){
        DDRC = 0xFF;                    /* PORTC as output*/

        /* main loop */
        while(1){
                PORTC=0x01;             /* Pulse PC1 ON*/
                timer_delay();          /* 20us delay */
                PORTC=0x00;             /* Pulse PC1 OFF */
                timer_delay();          /* 20us delay */
        }
}

void timer_delay(){
        TCNT0 = 0x5F;                   /* Load TCNT0 with the start value*/
        TCCR0 = 0x01;                   /* Timer0, normal mode, no pre-scalar */

        while((TIFR&0x01)==0);          /* Wait for TOV0 to roll over, i.e. overflow */
        TCCR0 = 0;                      /* set TCCR0 to 0 to stop the timer */
        TIFR = 0x1;                     /* Clear TOV0 flag*/
}
```

Internal Timer Interrupts:

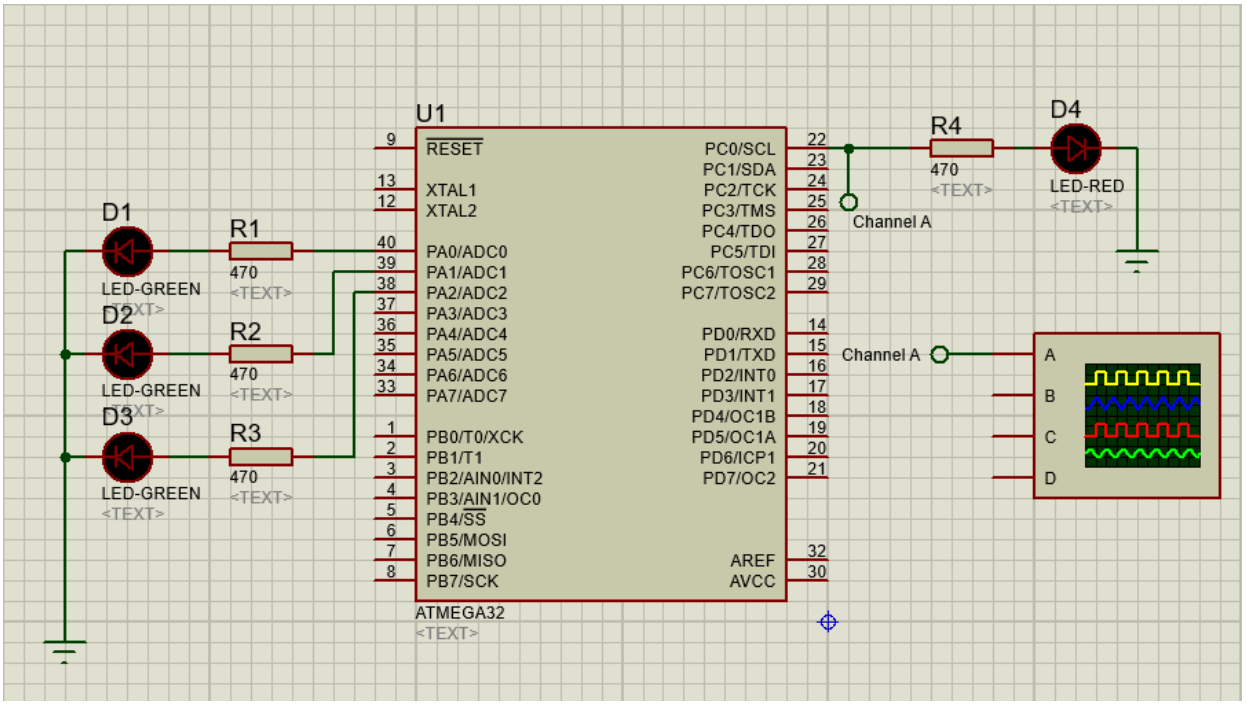
Internal interrupts as mentioned previously are interrupts triggered by internal devices such as Timers, ADC, UART, SPI etc. We will see how timers can be configured to produce interrupts when the timer overflows. The TIMSK register is used to do this setup

TIMSK: Timer / Counter Interrupt Mask Register

7	6	5	4	3	2	1	0
OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0

We have to set TOIE0 (Timer0 Overflow Interrupt Enable) bit in TIMSK register to set the timer0 interrupt, so that as soon as the Timer0 overflows, the controller jumps to the Timer0 interrupt routine.

Circuit Implementation:



Code Implementation:

```
/*
 * Timer_intr.c
 * Author : Michael Mureithi
 */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

/*Interrupt Service Routine for INT0*/
ISR(TIMER0_OVF_vect)
{
        PORTC ^= (1<<PC0); /* Toggle PORTC PC0 */
        TCNT0 = 0xB2;                /* At the end of each cycle: Reload start value to TCNT0 */
}


int main(void){
        /* Initialization */
```

```
        DDRA = 0x07;                /* Make PORTA as output PORT i.e. PA0, PA1, PA2*/
        DDRC = 1<<PC0;              /* Make PORTC as output PORT i.e. PC0*/
        DDRD = 0;                   /* PORTD as input */
        PORTD = 1<<PD2;             /* Activate pull up resistor high */

        /* Interrupt setup */
        TIMSK=(1<<TOIE0);           /* Enable Timer0 overflow interrupts */

        TCNT0 = 0xB2;               /* Load TCNT0, count for 10ms*/
        TCCR0 = (1<<CS02) | (1<<CS00);        /* Start timer0 with /1024 pre-scaler*/
        sei();                                /* Enable Global Interrupt */

        while(1){
                /* flashing LED code*/
                PORTA = 1<<PA0;
                 delay ms(200);
                PORTA = 1<<PA1;
                _delay_ms(200);
                PORTA = 1<<PA2;
                _delay_ms(200);
        }
}
```

3st Report: A Review of SPI and I2C

5 page (Excluding title page and reference page)
2 Weeks submission Deadline: Submission Date –
Include diagrams, pictures and flowcharts.
Give an example/sample

Lesson 10: PWM in ATmega32

Introduction

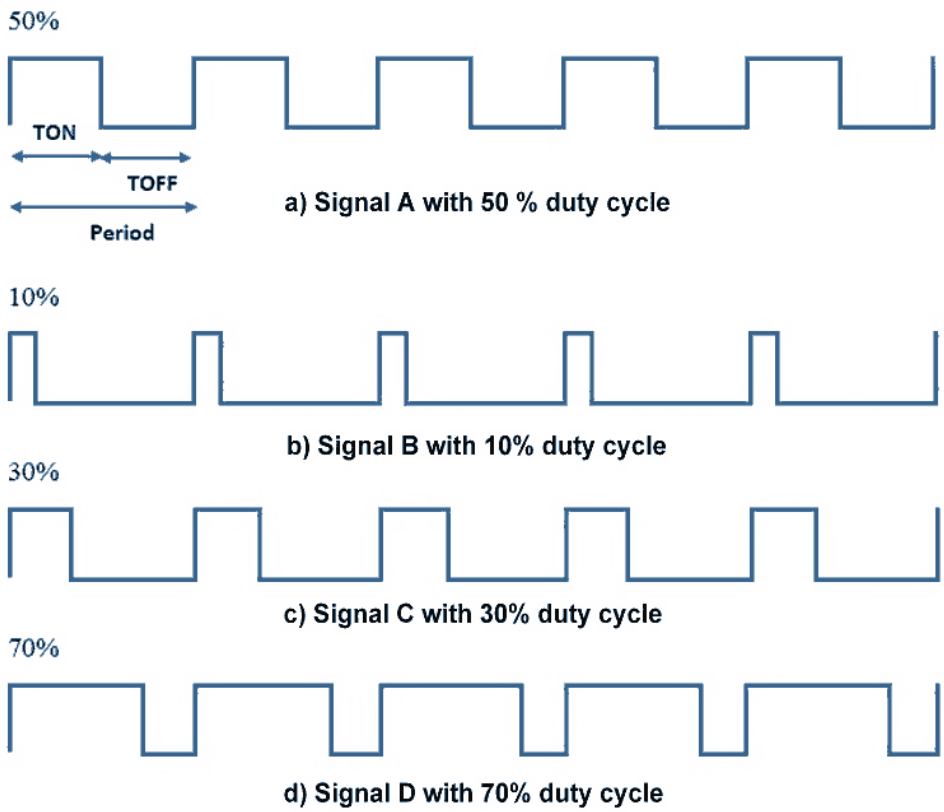
Pulse Width Modulation (PWM) is a technique by which width of a pulse is varied while keeping the frequency constant. This is a necessary feature that allows us to produce analogue outputs and control output devices. Let's take an example of controlling DC motor speed, more the Pulse width more the speed. Also there are application like, controlling light intensity by PWM.

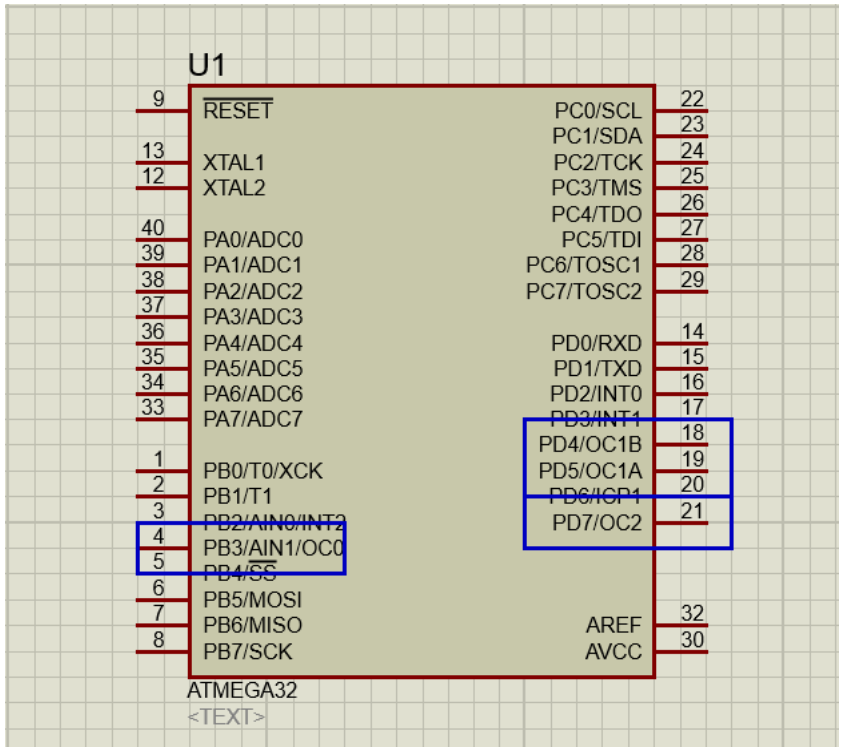
A period of a pulse consists of an ON cycle (5V) and an OFF cycle (0V). The fraction for which the signal is ON over a period is known as duty cycle.

Duty Cycle (In %) = \frac{T_O_N}{Total Period} * 100

E.g. Consider a pulse with a period of 10ms which remains ON (high) for 2ms.The duty cycle of this pulse will be D = 2ms / 10ms = 20%

Through PWM technique, we can control the power delivered to the load by using ON-OFF signal. Pulse Width Modulated signals with different duty cycle are shown below





ATmega has inbuilt PWM unit. As we know, ATmega has 3 Timers T0, T1 and T2 which can be used for PWM generation. Mainly there are two modes in PWM.

Fast PWM

Phase correct PWM

We need to configure Timer Register for generating PWM. PWM output will be generated on corresponding Timer's output compare pin (OCx) as shown in the figure above.

<u>Configuring Timer0 for PWM generation</u>

It is simple to configure PWM mode in Timer. We just need to set some bits in TCCR0 register.

TCCR0: Timer Counter Control Register 0

7	6	5	4	3	2	1	0
FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00

Bit 7- FOC0: Force compare match

Write only bit, which can be used while generating a wave. Writing 1 to this bit will force the wave generator to act as if a compare match has occurred.

Bit 6, 3 - WGM00, WGM01: Waveform Generation Mode

WGM00	WGM01	Timer0 mode selection bit
0	0	Normal
0	1	CTC(Clear timer on Compare Match
1	0	PWM, phase correct
1	1	Fast PWM

Bit 5:4 - COM01:00:

When WGM00:WGM01= 11 i.e. Fast PWM. Compare Output Mode

waveform generator on OC0 pin

COM01	COM00	Mode Name	Description
0	0	Disconnected	Normal port operation, OC0 disconnected
0	1	Reserved	Reserved
1	0	Non-inverted	Clear OC0 on compare match, set OC0 at TOP
1	1	Inverted PWM	Set OC0 on compare match, clear OC0 at TOP

2. When WGM00:WGM01= 10 i.e. Phase correct PWM. Compare Output Mode

COM01	COM00	Description
0	0	Normal port operation, OC0 disconnected
0	1	Reserved
1	0	Clear OC0 on compare match when up-counting, set OC0 on compare match when down –counting
1	1	Set OC0 on compare match when up-counting, set OC0 on compare match when down –counting

Bit 2:0 - CS02:CS00: Clock Source Select

These bits are used to select a clock source. When CS02: CS00 = 000, then timer is stopped. As it gets value between 001 to 101, it gets clock source and starts as the timer.

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer /counter stopped)
0	0	1	Clk (no pre-scaling)
0	1	0	Clk/8
0	1	1	Clk/64
1	0	0	Clk/256
1	0	1	Clk/1024
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

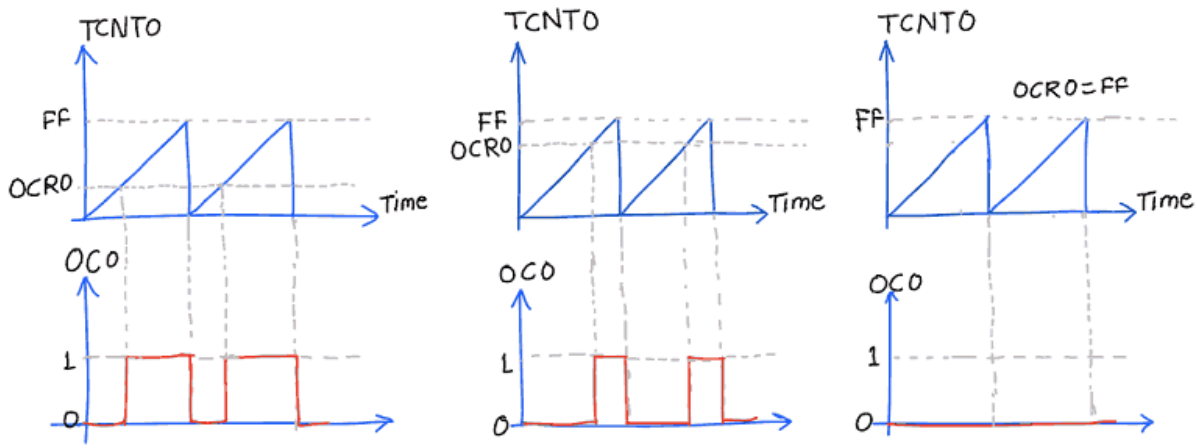
Fast PWM mode

To set Fast PWM mode, we have to set WGM00: 01= 11. To generate PWM waveform on OC0 pin, we need to set COM01:00= 10 or 11.

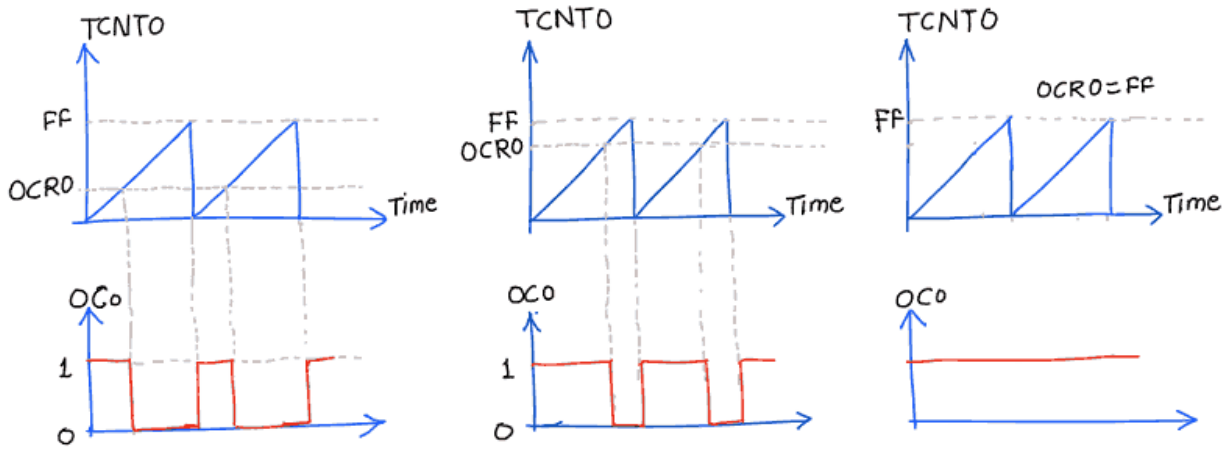
COM01:00= 10 will generate Noninverting PWM output waveform and COM01:00= 11 will generate Inverting PWM output waveform. See the fig.

```
void PWM_init(){

        /*set fast PWM mode with non-inverted output*/

        TCCR0 = (1<<WGM00) | (1<<WGM01) | (1<<COM01) | (1<<CS00);

        DDRB|=(1<<PB3);  /*set OC0 pin as output*/

}
```

Setting Duty cycle: we have to load value in OCR0 register to set duty cycle. 255 value for 100% duty cycle and 0 for 0% duty cycle. Accordingly, if we load value 127 in OCR0, Duty cycle will be 50%.



Inverted Fast PWM



Non-Inverted Fast PWM

<u>Nov-inverted Fast PWM</u>

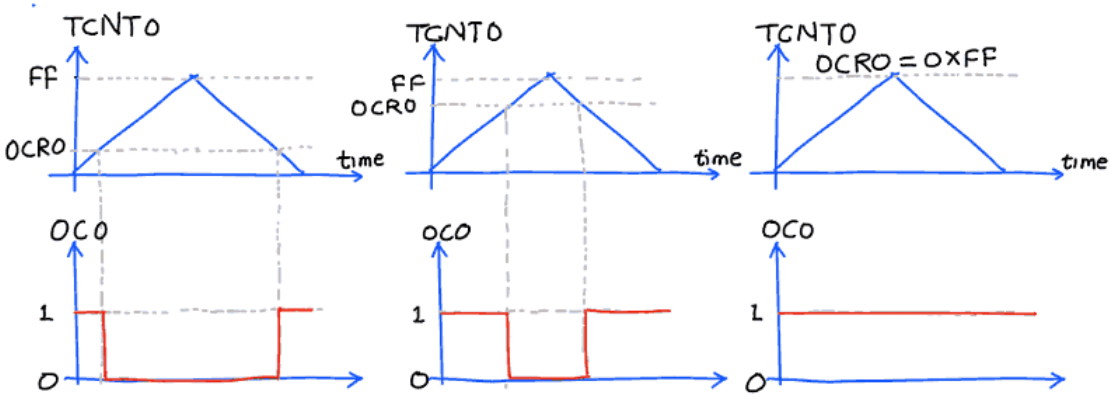
Advantage of using PWM mode in AVR is that it is inbuilt hardware unit for waveform generation and once we set the PWM mode and duty cycle, this unit starts generating PWM and controller can do other work.

<u>Phase correct PWM mode</u>

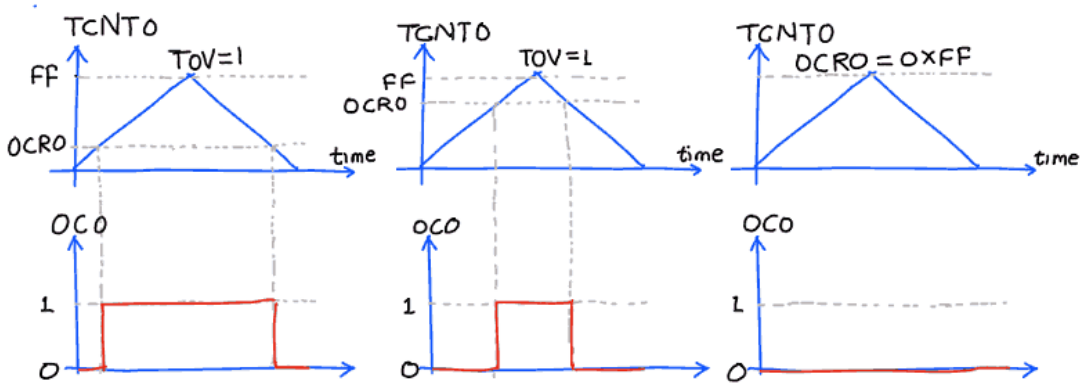
To set Phase correct PWM, we just have to set TCCR0 register as follow.

We can set output waveform as inverted or non-inverted. See the fig.

TCCR0 = (1<<WGM00) | (1<<COM01) | (1<<CS00);



Non-Inverted Phase correct PWM

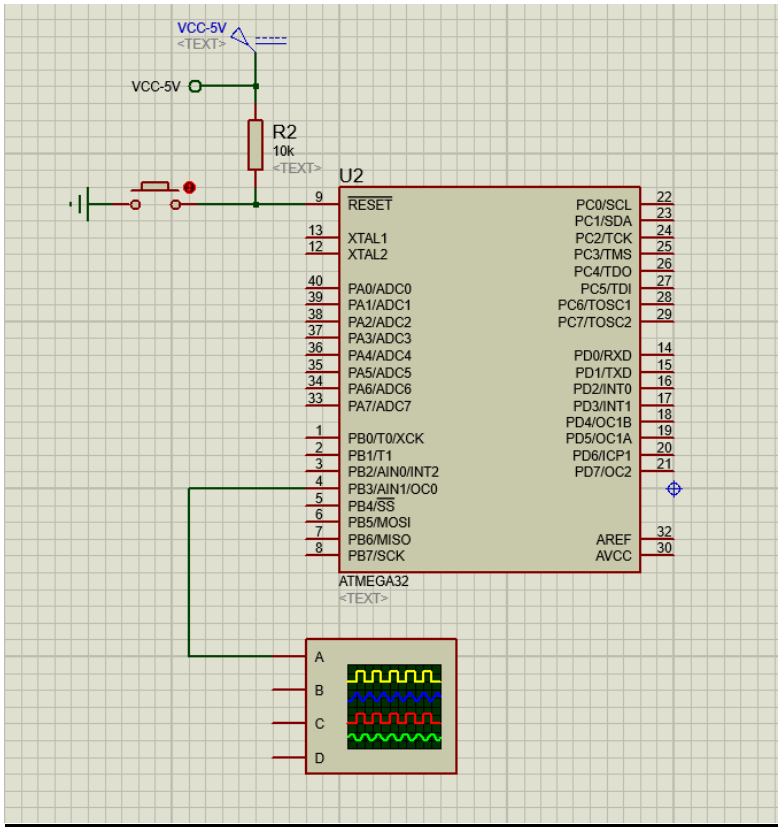


Inverted Phase correct PWM

Similarly, we can set PWM output on other three OCx pins using Timer1 and Timer2.

PWM output is somehow close to the Analog output. We can use it as analog output for generating sine wave, audio signals etc. it is also referred as DDS.

Circuit Implementation:

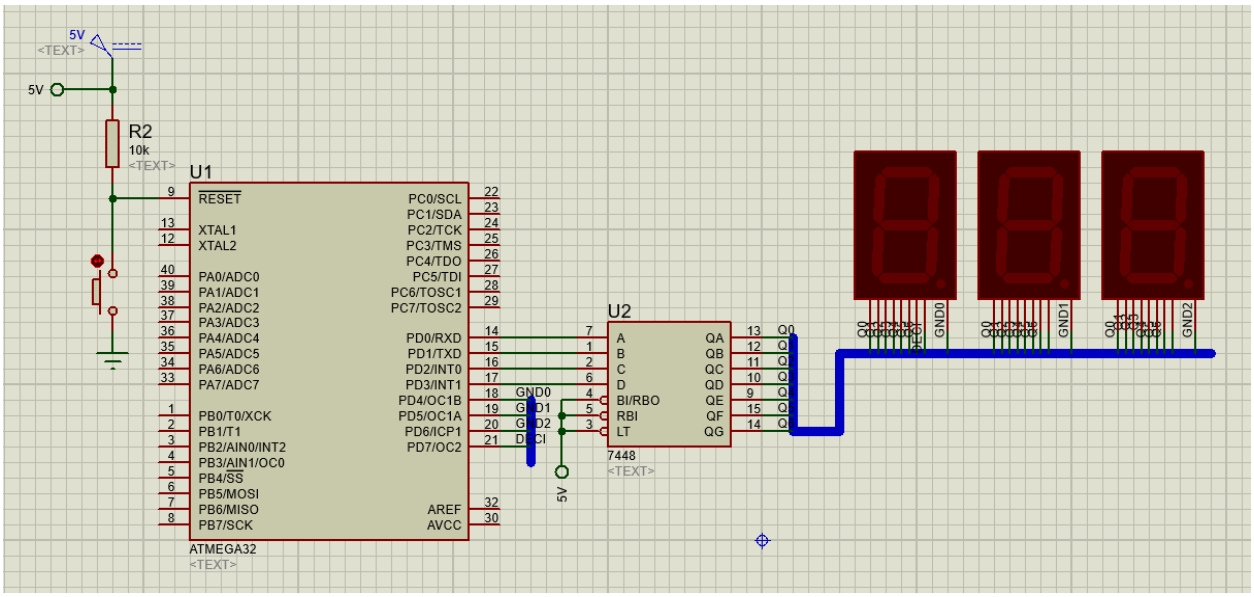


Code Implementation:

```
/*
 * PWM_simple.c
 *
*/
#define F_CPU 8000000UL
#include <avr/io.h>

void PWM_Init(){
	/*set fast PWM mode with non-inverted output*/
	// The PWM uses timer 0;
	TCCR0 = (1<<WGM00) | (1<<WGM01) | (1<<COM01) | (1<<CS00);
	DDRB|=(1<<PB3);  /*set OC0 pin as output*/
}

int main(void){

	PWM_Init();
	int cycle_value = 150;

	while(1){
		OCR0 = cycle_value;
	}
}
```

Advanced Lesson:

Going back to our earlier example using the 7-segment LED display, we noticed that it takes around 8 pins per 7-LED in order to operate it successfully. For situations where we need multiple 7SEG-LED, we need to find a more efficient solution that enables us to conserve the micro-controller pins which can be used for other applications. In order to do this we will use a BCD (Binary coded decimal) to seven segment decoder/driver. In our case we will use the 7448. This chip is able to take binary input eg. 1010(bin) and convert it to outputs for the 7 SEG-LED. The circuit below shows an implementation of the 7448 to run 3, 7 SEG-LEDs. Pins A, B, C and D are the binary input pins and Qa to Qg are connected to the 3, 7 SEG-LEDs using the bus.



Method of Operation:

After making the connection, it is now time to look at how we can light all 3 LEDs using just one 7448. This is done by lighting each LED, one at a time and switching between them very fast. The fast switching is done using a timer. Due to the fast switching, a phenomenon known as persistence of vision is created. Even though each LED is only lit for a short period of time, by performing this action very fast, it seems that they are lighting all at the same time. The switching between the LEDs is done through the use of the ground pins connected to PD4 to PD6. PD7 is used to light the decimal light on the first LED. Below is a sample code for running the above circuit.

The code below implements a simple count down timer that runs for 1 minute and 10 seconds. Timer0 is used to refresh the display of the 3, 7 SEG-LEDs after every 10ms using the ISR. After displaying each digit on its own LED, the program delays there for a millisecond or so in order to create the Persistence of Vision (POV) effect. In the main while loop, the time amount is decreased and then displayed again until the clock reaches 0 and the system flashes 0.

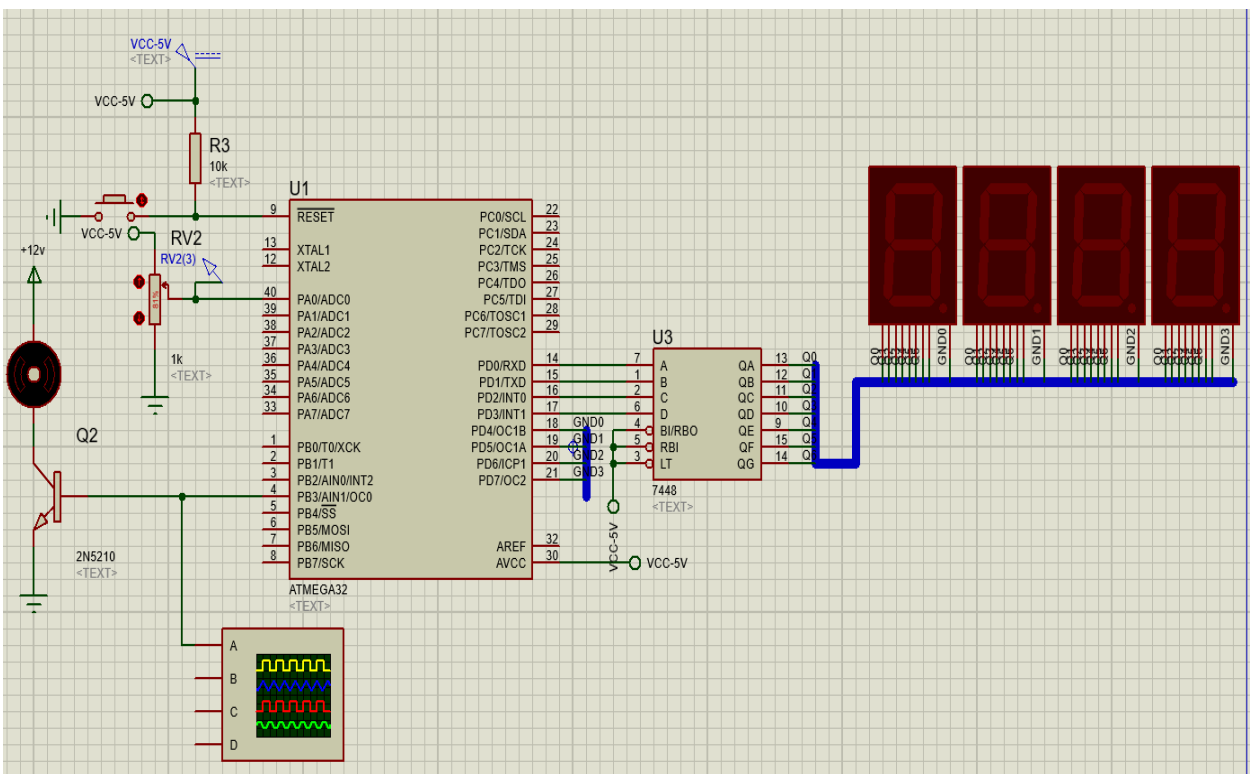
Code Implementation:

```
/*
 * BCD_7SEG.c
 * Author : Michael Mureithi
 */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdbool.h>

#define LED_Directions DDRD                    /* define LED Direction */
#define LED_Selection DDRC                     /* define LED Selection */
```

```c
#define LED_PORT1 PORTD                         /* define LED Display port */


char array[]={0,1,2,3,4,5,6,7,8,9};
int k,j,i;
int delayvalue=0;

/* setup the minutes and seconds values*/
int minutes = 1;
int seconds = 10;


/* A function to act as a persistence of vision delay*/
void vision_persist()
{
        int delay = delayvalue;
        while (0 < delay){
                _delay_us(1);
                --delay;
        }
}

/* At the end of each timer period, we display the contents of the 3 LEDs
        with some delay between each segment display
*/
ISR(TIMER0_OVF_vect)
{
        LED_PORT1 = 0xE0|array[i];
        vision_persist();

        LED_PORT1 = 0xD0|array[j];
        vision_persist();

        LED_PORT1 = 0xB0|array[k];
        vision_persist();
}

/* A function to separate a decimal number into its individual digits */
void SevenSeg_SetNumber(int num)
{
        k=num%10;
        num = num/10;

        j=num%10;
        num = num/10;

        i=num%10;
        num = num/10;
}

/* A function to reload the timer initial value after the timer overflows */
void sevseg_refreshDisplay(char refvalue)
{
        TIMSK=(1<<TOIE0);               /* Enable Timer0 overflow interrupts */
        TCNT0 = refvalue;               /* load TCNT0, count for 10ms*/
        TCCR0 = (1<<CS02) | (1<<CS00);  /* start timer0 with /1024 prescaler*/
}

int main(void)
{
        sei();
        LED_Directions = 0xff;          /* define port direction is output */
        LED_Selection = 0xff;           /* define port direction is output */
        LED_PORT1 = 0xff;
        sevseg_refreshDisplay(0xB1);/* set refresh rate of display (for 10ms set 0xB1) */
        delayvalue=1000;                /* set brightness level of 7 segment display */
        bool active = true;
        int count_down = (minutes*60)+seconds;
```

```
        while(1)
        {
                if(active){
                        for(int i=count_down;i > 0;i--){
                                int secs = i%60;
                                int mins = i/60;
                                int disvalue = (mins*100)+secs;
                                SevenSeg_SetNumber(disvalue);
                                _delay_ms(1000);
                        }
                        active = false;
                }
                else{
                        SevenSeg_SetNumber(0);
                }
        }
}
```

A more advanced example that combines the elements of Timers, ADC and PWM is the control of a DC motor using a potentiometer. The circuit is as shown below. We have a potentiometer connected to PA0 (ADC0) and a DC motor connected through a transistor (2N5210) to PB3 (OC0). The value of the ADC conversion is to be displayed in the 4, 7 SEG-LEDs, and based on this value, a PWM signal should be generated to drive the DC motor on PB3.



Code Implementation:

```
/*
 PWM DC motor control
*/
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdbool.h>

#define LED_Directions DDRD                    /* define LED Direction */
#define LED_Selection DDRC                     /* define LED Selection */
#define LED_PORT1 PORTD                                /* define LED Display port */


char array[]={0,1,2,3,4,5,6,7,8,9};
int i,j,k,l;
int delayvalue=0;

/* setup the minutes and seconds values*/

void display_Init(){
         LED_Directions = 0xff;                /* define port direction is output */
         LED_Selection = 0xff;                 /* define port direction is output */
         LED_PORT1 = 0xff;
}

/* A function to act as a persistence of vision delay*/
void vision_persist()
{
         int delay = delayvalue;
         while (0 < delay){
                  _delay_us(1);
                  --delay;
         }
}

/* At the end of each timer period, we display the contents of the 3 LEDs
         with some delay between each segment display
*/
ISR(TIMER2_OVF_vect)
{
         LED_PORT1 = 0xE0|array[i];
         vision_persist();

         LED_PORT1 = 0xD0|array[j];
         vision_persist();

         LED_PORT1 = 0xB0|array[k];
         vision_persist();

         LED_PORT1 = 0x70|array[l];
         vision_persist();

}

/* A function to separate a decimal number into its individual digits */
void SevenSeg_SetNumber(int holder)
{
         int num = holder;
         l=num%10;
         num = num/10;

         k=num%10;
         num = num/10;

         j=num%10;
         num = num/10;

         i=num%10;
         num = num/10;
}

/* A function to reload the timer initial value after the timer overflows */
void sevseg_refreshDisplay(char refvalue)
{
         TIMSK=(1<<TOIE2);           /* Enable Timer2 overflow interrupts */
         TCNT2 = refvalue;           /* load TCNT2, count for 10ms*/
```

```
        TCCR2 = (1<<CS22) | (1<<CS21) | (1<<CS20);  /* start timer2 with /1024 prescaler*/
}


void ADC_Init(){
        DDRA = 0x0;                             /* Make ADC port as input */
        ADCSRA = 0x87;                  /* Enable ADC, fr/128  */
        ADMUX = 0x40;                   /* Vref: Avcc, ADC channel: 0 */
}

int ADC_Read(int channel){
        int Ain,AinLow;

        ADMUX = ADMUX|(channel);    /* Set input channel to read */

        ADCSRA |= (1<<ADSC);                            /* Start conversion */
        while((ADCSRA & (1<<ADIF)) == 0);     /* Monitor end of conversion interrupt */

        _delay_us(10);
        AinLow = (int)ADCL;                     /* Read lower byte*/
        Ain = (int)ADCH*256;            /* Read higher 2 bits and Multiply with weight */
        Ain = Ain + AinLow;
        return(Ain);                            /* Return digital value*/
}


void PWM_Init()
{
        /*set fast PWM mode with non-inverted output*/
        TCCR0 = (1<<WGM00) | (1<<WGM01) | (1<<COM01) | (1<<CS00);
        DDRB|=(1<<PB3);  /*set OC0 pin as output*/
}

int main(void)
{
        sei();
        PWM_Init();
        display_Init();
        sevseg_refreshDisplay(0xB1);/* set refresh rate of display (for 10ms set 0xB1) */
        delayvalue = 1500;
         int current_value = 0;
         int buffer = 0;

        ADC_Init();
        while(1){
                current_value = ADC_Read(0);/* Read ADC channel 0 */
                buffer = current_value*0.25;
                OCR0 = buffer;
                SevenSeg_SetNumber(buffer);
        }
}
```

The code above is just the combination of the concepts in ADC, Timers and PWM that we have done previously. It demonstrates that quite complex system can be achieved by cleverly combining these features and harmonizing them such that they work together.